

ENGINEERING PLAN · AGENTIC ORCHESTRATION · P2 STUDIO

LOONA — OS personal self-hosted con cara de Zoey, paleta propia y más perillas

Plan de ingeniería fullstack: arquitectura, flujos, grafos, gobernanza y harness para Grok + Antigravity CLI (Agy) + Claude Code + Codex. Hardware real: iMac Intel i5-6500 / 24 GB / Monterey 12.7.6.

Campo	Valor
Producto	LOONA — no se llama Jarvis. No es Zoey OS.
Inspiración visual	Zoey OS World View: cabeza de partículas 3D + Knowledge View + companions
Diferencia visual	Paleta teal #5EEAD4 → violeta #7C5CFF (Zoey es ámbar). Más config.
Ref de video	Downloads/ScreenRecording_08-12-2026 11-06-01_1.MP4 (27s)
Lead	Grok — orquesta, verifica, único DONE global
Agy / Antigravity	wB:p1 — HUD / World View · write hud/ refs/ assets/
Claude Code	wB:p2 — runtime / API / OpenClaw · write runtime/
Codex	wB:p3 — spec / schema / memoria · write docs/ identity/ config/
DoD	8 pruebas en docs/GOVERNANCE.md. Loops hasta verde.

Contenido

- 1. Misión, DoD y por qué no instalamos Zoey
- 2. DNA visual (Zoey → LOONA) y superficie de configuración
- 3. Arquitectura de contenedores y envelope de hardware
- 4. Flujos: datos, chat, secretos
- 5. Grafo de agentes, contratos y DAG
- 6. Harness, gobernanza y máquina de estados
- 7. Ingeniería del HUD (partículas, config, knowledge)
- 8. Ingeniería del runtime (API, cerebro, OpenClaw)
- 9. Loop 1 — briefs por pane (para no pelearse)
- 10. Verify y criterio de stop

1. Misión, DoD y por qué no instalamos Zoey

Zoey OS es un SaaS cerrado (zoeyos.com): World View 3D, companions, Knowledge Base, skills, 1 000+ integraciones, automations. El video de referencia muestra exactamente eso: fondo negro, cabeza de puntos ámbar, label ZOEY, flechas, botón KNOWLEDGE VIEW, rail de tareas y panel KARMA/PULSE. No es open source. Instalar Zoey sería cuenta + trial + depender de su nube. Eso rompe el contrato del studio (self-hosted, keys nuestras, free-first) y no corre bien como “dueño” en un iMac Intel de 2015.

LOONA clona el **lenguaje visual y la UX** (cara de partículas, world view, knowledge, config, companions), cambia el color, expone **más perillas** en JSON schema, y corre 100% en 127.0.0.1 con Grok/Claude/Gemini/Ollama. El cerebro 24/7 puede ser OpenClaw; el cuerpo visual es nuestro HUD.

Definition of Done (8 pruebas, todas o nada)

#	Prueba	Evidencia
1	HUD en http://127.0.0.1:8766	curl / o screenshot
2	Cabeza de partículas teal→violeta, label LOONA	refs/hud/world.png
3	Config ≥ 12 controles reales ligados al schema	refs/hud/config.png
4	Knowledge View muestra SOUL.md	texto visible en HUD
5	Chat del HUD responde con un modelo	turno + log
6	Runtime health o OpenClaw doctor verde	GET /api/health
7	Guardrail bloquea .env y publish	respuesta de rechazo
8	STATUS.md con hora MTY	docs/STATUS.md

Criterio legal / ético de inspiración

Se permite: paleta propia, geometría de partículas original, layout (centro + rails + chat). Se prohíbe: extraer CSS/JS de zoeyos.com, usar su wordmark, reclamar compatibilidad. Zoey es la referencia de UX; LOONA es implementación limpia.

2. DNA visual (Zoey → LOONA) y superficie de configuración

Del screen recording (27.5 s, 1320×2868, 12 ago 2026) se extrae el sistema visual:

Pieza Zoey (ref)	Qué hace	LOONA (delta)
Cabeza point-cloud ámbar	Identidad viva del OS, rota	Misma idea, gradiente teal→violeta, morph face/sphere/moon
Label ZOEY + flechas	Identidad + switch de vista	Label LOONA. Flechas ciclan companions
KNOWLEDGE VIEW	Memoria visible	Panel que lee identity/*.md vía /api/knowledge
Rail de tareas (ads, CTR...)	Trabajo en curso	Activity widget + companions Brain/Memory/Ops
Chat KARMA / PULSE	Conversación continua	Chat dock redimensionable, pop-out fase 2
Config View (guía oficial)	Hasta 20 companions, interviews	JSON schema versionable, ≥12 knobs, sin interviews mágicos

Más config que Zoey (día 1), todo en config/loona.schema.json:

- world:** particleCount, paletteFrom/To, headMorph, rotationSpeed, bloom, showOrbs
- voice:** wakeWord=loona, mode typed-only|wake|ptt, stt, tts
- brain:** provider, model, baseUrl, dailyTokenCap, temperature
- memory:** soulPath, retentionDays, compound
- guardrails:** sandbox, allowPublish, allowSecretRead, computer-use, askOnIrreversible
- channels:** telegram, hud, bind 127.0.0.1

- **hud:** port, chatDefaultWidth, showActivity, knowledgeOpen
- **companions[]:** id, name, role, color, model, skills (máx 8 en este iMac, no 20)

3. Arquitectura de contenedores y envelope de hardware

Cuatro contenedores lógicos en un solo host. Nada de Kubernetes. El i5-6500 orquesta; el modelo pesado vive en la nube ya pagada o en Ollama 3B.

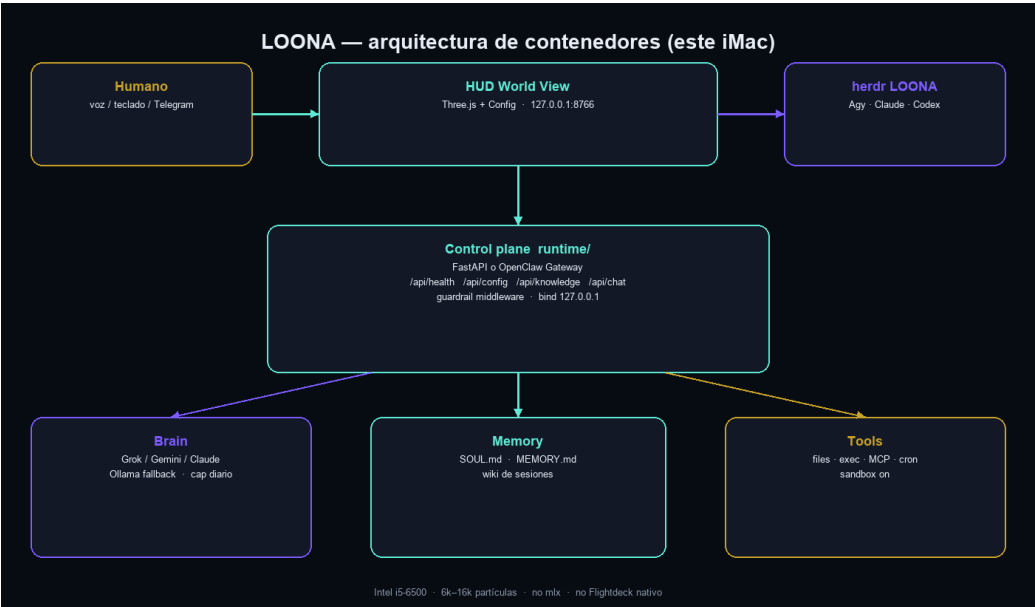


Figura 1. Contenedores: HUD, control plane, brain, memory, tools.

Constraint	Implicación de diseño
CPU 4c / 2015, HD 530	HUD: Points de Three.js, 6k–16k partículas. Sin bloom caro. Sin mlx-audio.
24 GB RAM	Ollama 3B–7B máx. No servir 32B. No Electron+Chrome+Ollama 14B a la vez.
macOS 12.7.6	Evitar Swift reciente / Flightdeck nativo. Browser localhost es el shell.
Node 24 + Python 3.14	OpenClaw viable. Si 3.14 rompe wheels, runtime en 3.12 o Node.
Ollama instalado, daemon down	Claude Code lo levanta o degrada a cloud key.

4. Flujos: datos, chat, secretos

4.1 Secuencia de un turno

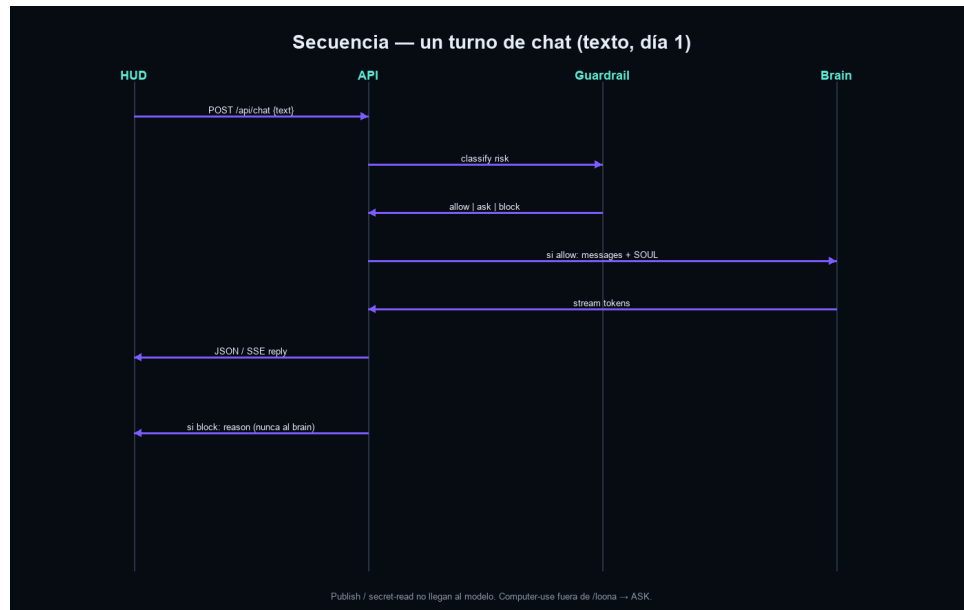


Figura 2. POST /api/chat → guardrail → brain → stream. Block no toca al modelo.

4.2 Datos y secretos



Figura 3. Repo sin keys. RAM con keychain. Egreso solo al provider elegido.

```

POST /api/chat
→ classify(text): safe | ask | block
→ block: {error:'guardrail', rule:'allowSecretRead|allowPublish'}
→ ask: {need_human:true, action:...}
→ allow: messages = [SOUL, MEMORY, history, user] → provider
  
```

5. Grafo de agentes, contratos y DAG

Un write-path por agente. Cuatro briefs idénticos es el anti-patrón del studio. Los contratos son HTTP + JSON schema + STATUS.md — no reuniones.

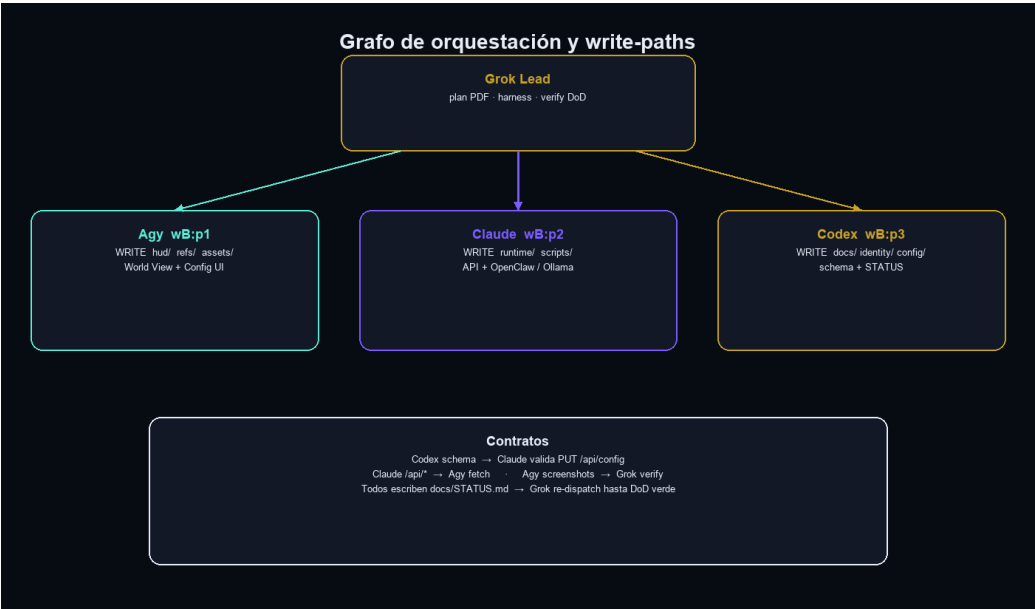


Figura 4. Grok despacha. Agy pinta. Claude sirve. Codex especifica.

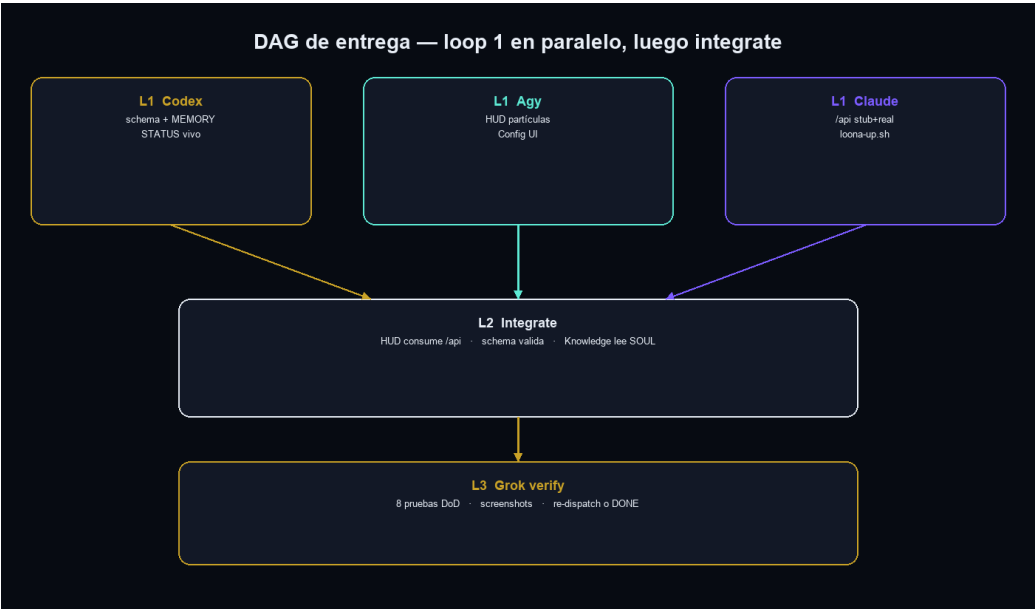


Figura 5. Loop 1 en paralelo (L1). Integrate (L2). Verify (L3).

Contrato	De	A	Forma
loona.schema.json	Codex	Claude + Agy	archivo en config/
GET/PUT /api/config	Claude	Agy	HTTP 127.0.0.1
GET /api/knowledge	Claude	Agy	text/markdown JSON
POST /api/chat	Claude	Agy	JSON
refs/hud/*.png	Agy	Grok	archivo
docs/STATUS.md	todos	Grok	append-only

6. Harness, gobernanza y máquina de estados

Canónico en disco: docs/GOVERNANCE.md, docs/HARNESS.md, docs/ROLES.md, identity/SOUL.md. El harness existe para que un agente idle no se quede esperando permiso. Si su checklist local no está verde, vuelve al siguiente ítem. Si un contrato ajeno no existe, implementa stub.



Figura 6. IDLE → READ → BUILD → EVIDENCE → (loop | BLOCKED | VERIFY).

Stop condition

El loop termina cuando Grok (no Agy, no Claude, no Codex) marca las 8 pruebas DoD. Un HUD bonito sin /api/health no es done. Un API sin cabeza de partículas no es done. NEED_HUMAN solo por login/MFA/pago/ToS o si OpenClaw pide un wizard interactivo en el pane.

7. Ingeniería del HUD (Agy / Antigravity)

- Vanilla HTML/CSS/JS + Three.js desde CDN. Sin React el día 1. Sin Electron.
- world.js: BufferGeometry + Points. Posiciones = esfera perturbada (simplex barato o noise 3D).
- Color por vértice: mix(paletteFrom, paletteTo, y). Label canvas/sprite "LOONA".
- FPS watchdog: si < 24 durante 2 s, bajar particleCount 20% y persistir via PUT /api/config.
- Vistas: World | Knowledge | Config. Teclas: 1/2/3, Esc cierra paneles, [] chat width.
- Config: un control por propiedad del schema (range, color, select, toggle). No hardcodear.
- Si /api/* no está: fallback fetch a /config/loona.default.json por static server.

```
hud/  
index.html  
css/loona.css # tokens --bg --teal --violet  
js/world.js # particulas + orbs  
js/config-view.js # liga schema  
js/chat.js # POST /api/chat  
js/knowledge.js # GET /api/knowledge
```

8. Ingeniería del runtime (Claude Code)

Prioridad A: OpenClaw onboard --install-daemon si el wizard no se atasca en Monterey. Prioridad B (más fluida en este iMac): FastAPI en runtime/app.py sirviendo HUD estático y los 4 endpoints. OpenClaw se puede enchufar después como provider.

Endpoint	Contrato
GET /api/health	{ok:true, name:"LOONA", provider, particles}
GET /api/config	objeto válido contra loona.schema.json

Endpoint	Contrato
PUT /api/config	merge + validar; 400 si viola schema
GET /api/knowledge	{soul, memory} markdown
POST /api/chat	{text} → {reply} o {error, rule}

Cerebro: LOONA_PROVIDER + key de entorno. Orden de fallback: XAI_API_KEY → GEMINI_API_KEY → ANTHROPIC_API_KEY → Ollama localhost. Guardrail regex: .env, api_key, sk-, publish, instagram.com/p/, rm -rf /.

9. Loop 1 — briefs por pane (para no pelearse)

Estos briefs se pegan tal cual vía herdr agent prompt <pane_id>.

Agy wB:p1 — HUD

Eres Agy/Antigravity en /Users/imac/loona. WRITE solo hud/ refs/ assets/.
 Lee identity/SOUL.md docs/GOVERNANCE.md docs/HARNESS.md docs/ROLES.md.
 Construye World View tipo Zoey (refs/zoey/*.jpg) con paleta teal→violeta,
 label LOONA, Knowledge View, Config ≥12 knobs, chat panel.
 Three.js CDN. 6k-16k partículas. Loop HARNESS hasta tu checklist verde.
 Screenshot refs/hud/world.png y config.png. STATUS.md al cerrar cada turno.
 NO toques runtime/ ni identity/SOUL.md. NO esperes a Claude: stubea /api.

Claude Code wB:p2 — runtime

Eres Claude Code en /Users/imac/loona. WRITE solo runtime/ y scripts/loona-up.sh.
 Lee SOUL, GOVERNANCE, HARNESS, ROLES, config/loona.schema.json.
 Levanta control plane 127.0.0.1 (FastAPI sirviendo hud/ + /api/*).
 OpenClaw solo si onboard es no-interactivo; si pide wizard, STATUS NEED_HUMAN
 y sigue con FastAPI. Chat + guardrails. Loop hasta checklist runtime verde.
 NO edites hud/css ni world.js.

Codex wB:p3 — spec

Eres Codex en /Users/imac/loona. WRITE solo docs/ identity/ config/
 (excepto no reescribir SOUL.md ni GOVERNANCE.md salvo typos).
 Completa MEMORY.md, API.md, alinea ARCHITECTURE.md al código que vaya apareciendo,
 mantén STATUS.md. Schema ya existe: extiende companions/skills si hace falta.
 Loop HARNESS. NO implementes Three.js ni el server.

10. Verify y criterio de stop

- Grok corre las 8 pruebas en orden. Falla = re-prompt SOLO al dueño del write-path.
- Si Agy pinta ámbar o escribe ZOEY/JARVIS: rechazo visual, no “casi”.
- Si Claude expone 0.0.0.0: rollback a 127.0.0.1.
- Si Codex y el código divergen: gana el código, Codex actualiza ARCHITECTURE.
- Stop = DoD verde + STATUS final. Entonces y solo entonces se habla de Telegram/voz wake.

Orden de ataque ahora

1) Este PDF y los markdown canónicos ya están en disco. 2) Grok despacha los tres panes en paralelo (L1). 3) Integrate L2 cuando existan hud/index.html y runtime health. 4) Verify L3. Re-loop. 5) Voz wake / Telegram = fase 2, no bloquean el DoD del HUD+API.

Canónicos: identity/SOUL.md · docs/GOVERNANCE.md · docs/HARNESS.md · docs/ROLES.md · config/loona.schema.json · refs/diagrams/*.png · este PDF.